Design a Car Rental System

Resources

- 9. LLD of Car Rental System (Hindi) | ZoomCar Low Level Design | System Design Interview Question
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/carrental · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding

Problem Statement

Design a detailed low-level model for a Car Rental System, including all relevant classes and design patterns covered earlier.

Flow and Requirement Classification

- User comes to care rental system.
- Select a particular Store based on its Location.
- Search for Vehicles based on vehicle type(like "Four Vehicle", "Two Wheeler" etc.) and its availability.
- Reserve the selected vehicle.
  a. Should handle concurrency, as 2 users try to reserve the same vehicle for same time.
- Start the trip.
- Submit the vehicle.
- Bill is generated against the reservation.
  a. We can have multiple bill strategies for generating the bill, like daily Bill strategy or hourly bill strategy etc.

- Payment is made against the Bill.
    a. We can have multiple payment strategies for processing the payment like UPI, Card etc.

Based on above understanding, we identified the below major objects involved in this design:

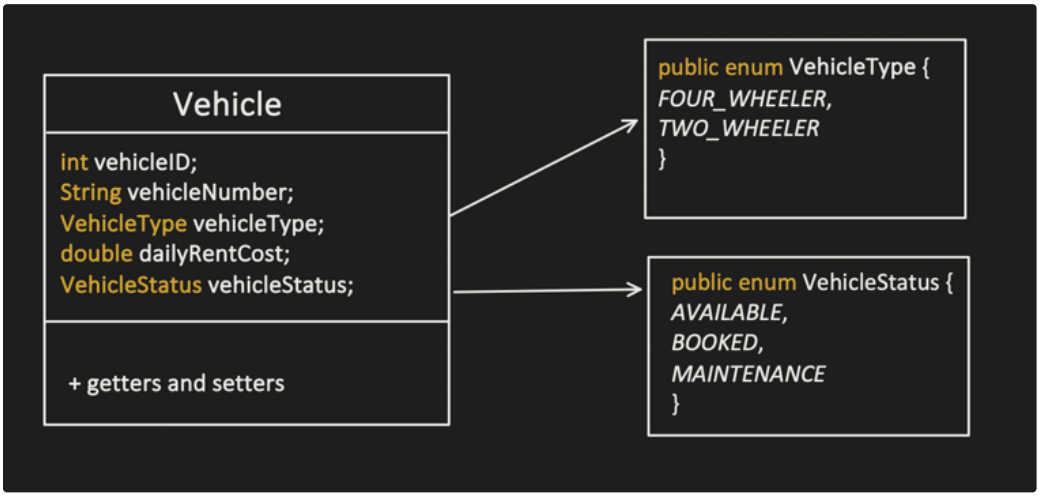
- Vehicle
- Store
- Reservation
- User
- Bill
- Payment

Lets start creating the UML:

1. `Vehicle`, `VehicleStatus` and `VehicleType`

- Vehicle class represents the basic building block of the Car Rental System.
- It comprises all the necessary details about the vehicle that is available for renting out, like vehicle number, type, model, odometer readings, date of manufacture and its rental charge.
- `VehicleStatus` and `VehicleType` help track and store more details about the `Vehicle` in use.

Concept && Coding



Vehicle object is not intelligent, its just a POJO. We need some other intelligent object, which can manage Vehicle inventory, like:

- Maintain overall vehicles list.
- Maintain booked vehicles list.
- Add vehicle
- Remove vehicle
- Check for vehicle availability

  etc.

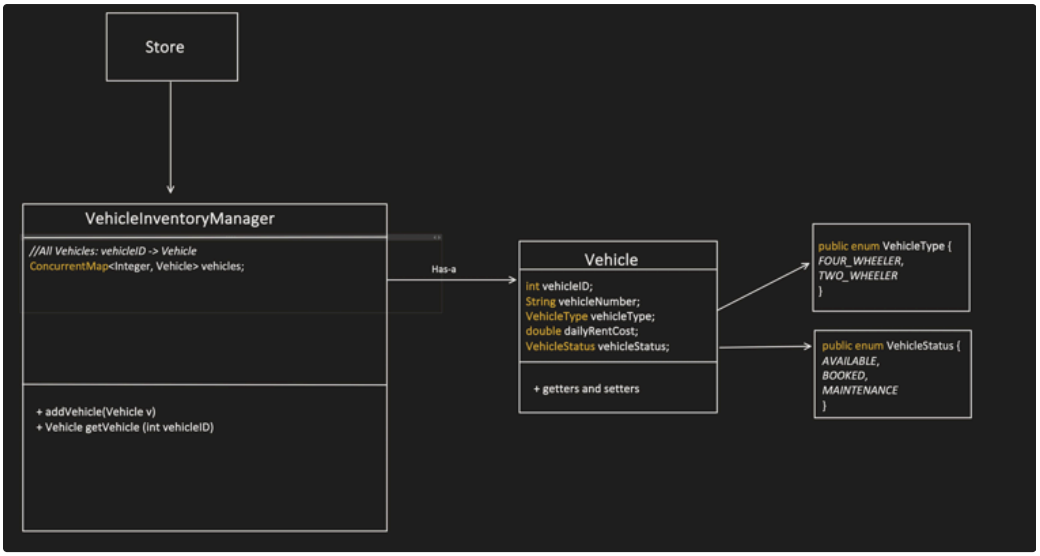
  That's where VehicleInventoryManager comes into the picture.

  Note:
  each store has their own VehicleInventoryManager, so it is not a singleton class

2. `VehicleInventoryManager`

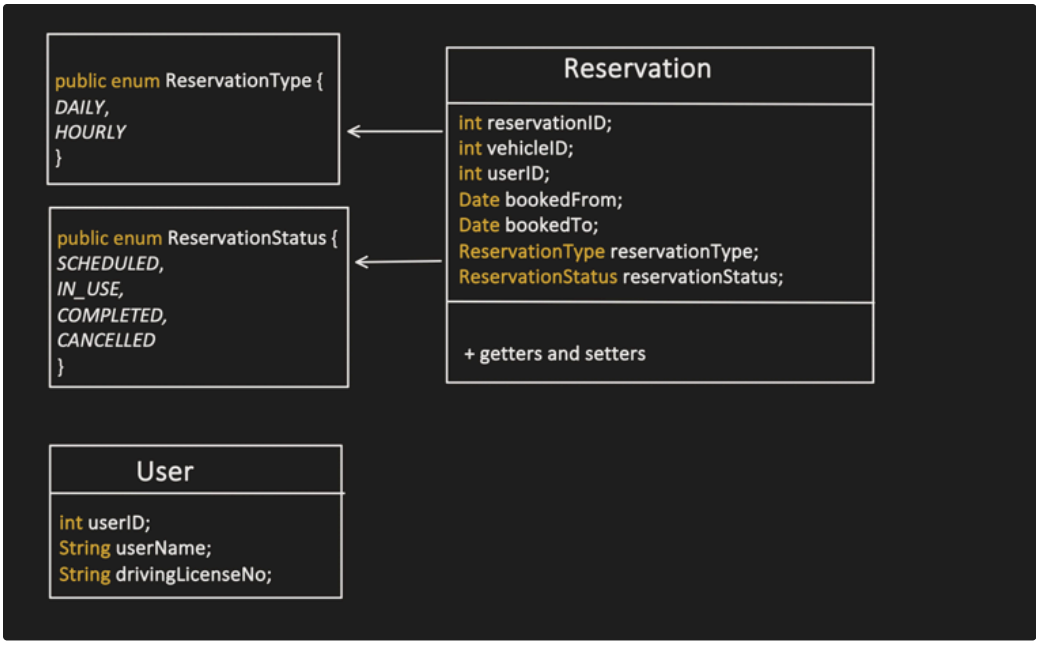
We will enhance the capability of this VehicleInventoryManager later when Reservation capability will get added.

Concept && Coding



3. `Reservation`

- Reservation, does not need full Vehicle and User details, so we can keep their ids itself in it.



Now, Reservation is not smart enough to handle capabilities like:

- Maintaining the List of Reservations.
- Fetch particular reservation details.
- Creation of reservation, changing the reservation status.
- Cancellation of reservation, changing the reservation status.
- Remove the reservation

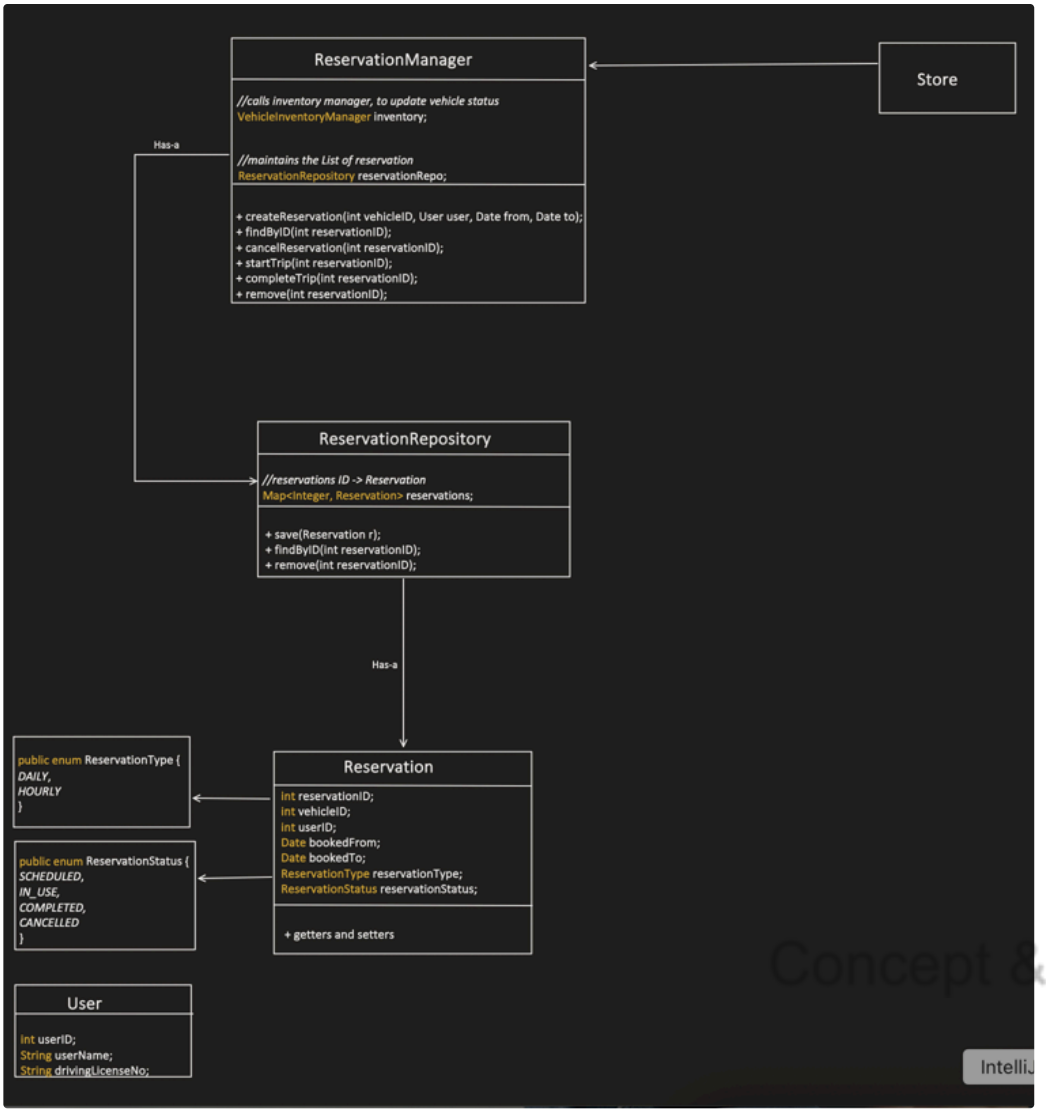
Concept && Coding

Etc.

That's where ReservationManager comes into the picture.

Note:
each store has their own ReservationManager, so it is not a singleton class

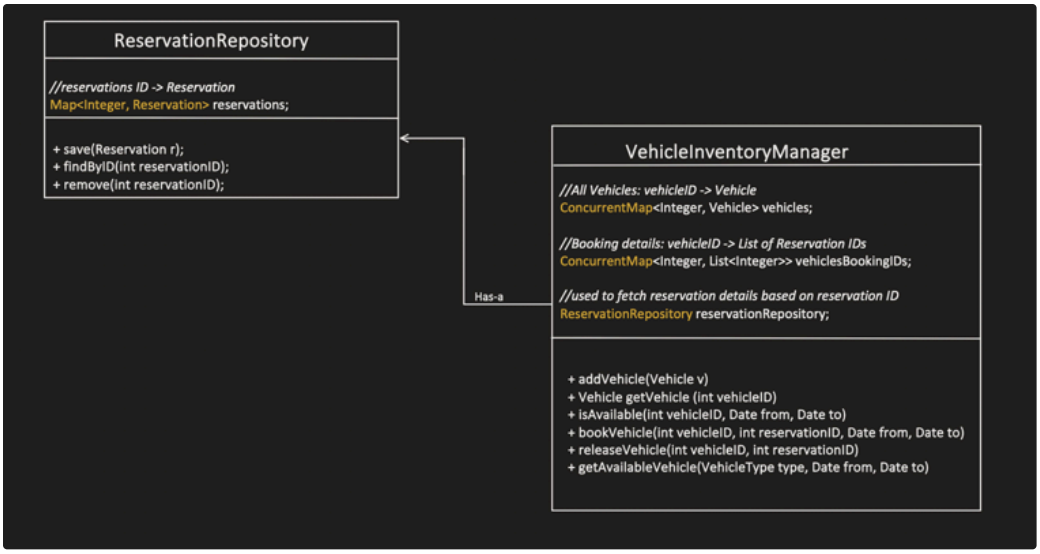
4. `ReservationManager`



Need to handle:

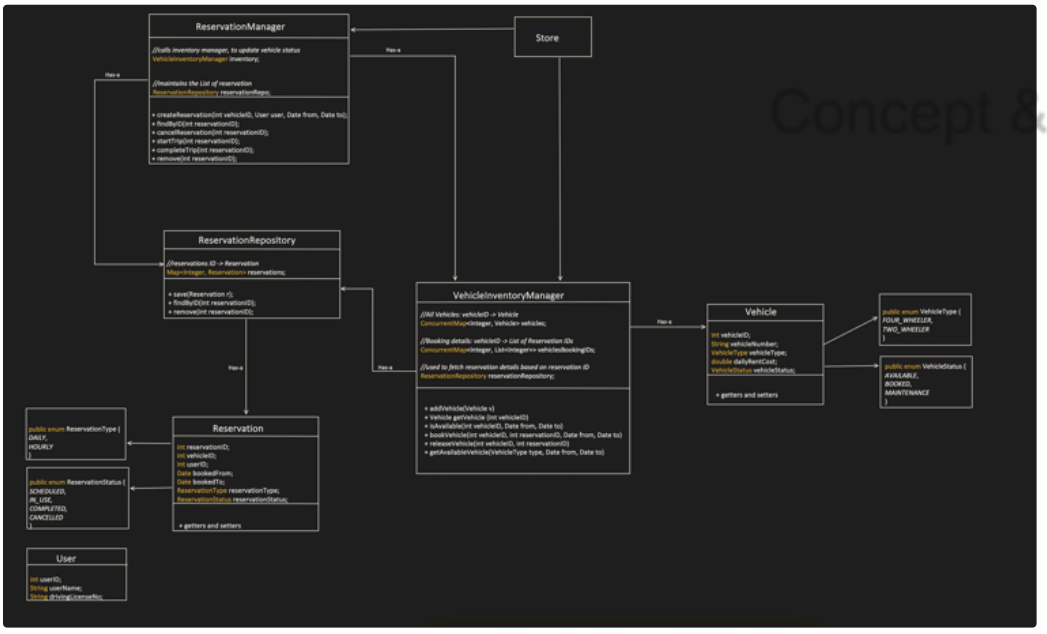
Create Reservation should only be successful, when vehicle is available on that particular date.

Whats the right place to hold the vehicle booking information. Its "VehicleInventoryManager"



Note:

I created a ReservationRepository to break the circular dependency between ReservationManager and VehicleInventoryManager.



Another thing which need to be handle is: Concurrency

2 important things to consider while handling the concurrency:

1st: Collections like List of vehicles, List of Booked vehicles should be able to handle concurrency. So that our underlying data structure should be consistent when multiple threads

try to read or write in it.

2nd: Need capability, to put lock per Vehicle, this is needed for achieve atomicity.

For example: assume both thread1 and thread2 running concurrently, below scenario will cause an issue:

Thread1:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Thread2:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Concept && Coding

Now, even though our collections are concurrent (only 1 thread can touch it at a time), still above scenario will cause an issue and both the

thread will be able to book the same vehicle for same slot.

To handle it, we need to bring lock per vehicle:

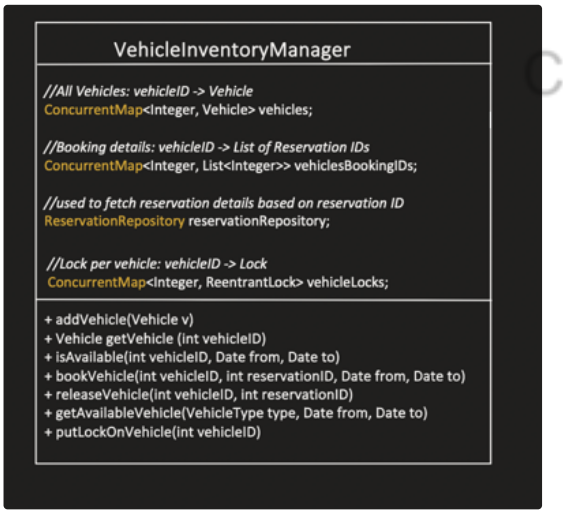
Thread1:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- **Put a lock on V1**
- **Check for availability again**
- Try to reserve this particular vehicle(V1), change the status of vehicle
- Update the List of Booked vehicles
- **Release lock**

Thread2:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- **Put a lock on V1**
- **Check for availability again**
- Try to reserve this particular vehicle, change the status of vehicle
- Update the List of Booked vehicles
- **Release lock**

Now, one thread say Thread2 will wait for the lock to release on V1.
When Thread1, release the lock, then Thread2 will proceed and check for availability again, but this time it will not be able to reserve as its not available.

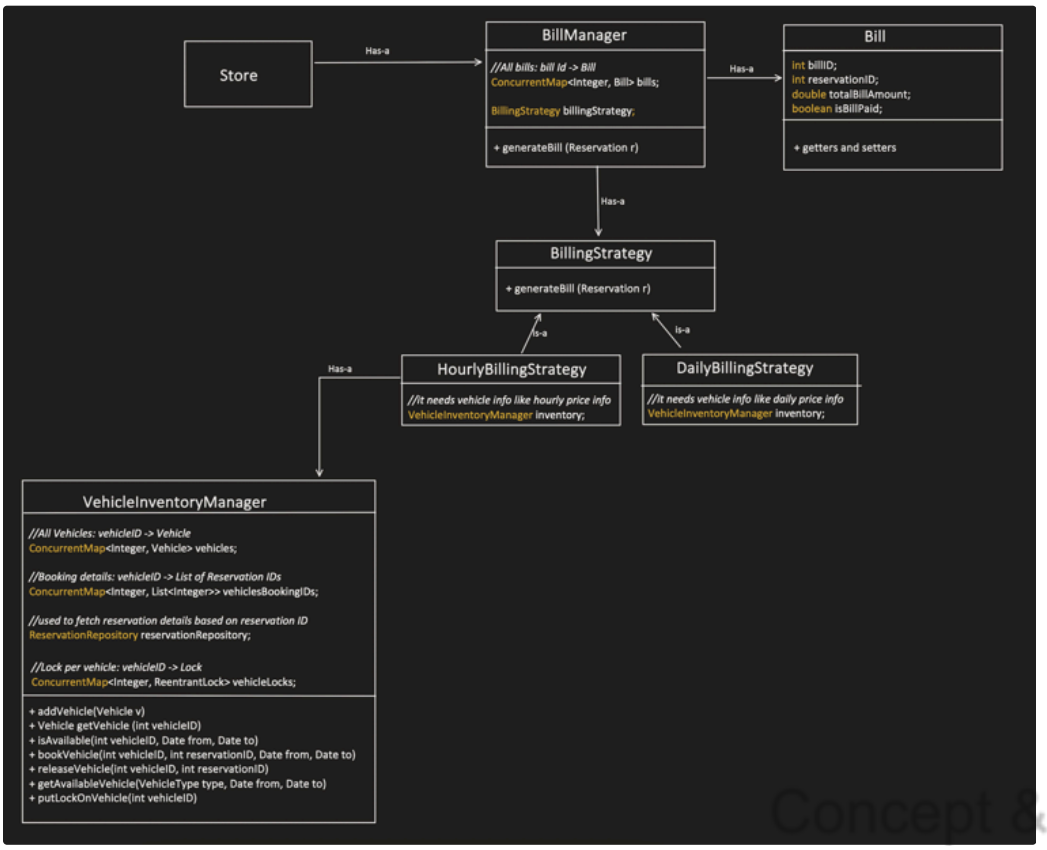


Concept && Coding

5. `Bill`

- Bill hold info like Reservation id like this bill is associated with which reservation. We don't need full object.
- Also, Bill is just a POJO class which is not smart enough. So we need:
  BillManager, which handles all the bills and their lifecycle.

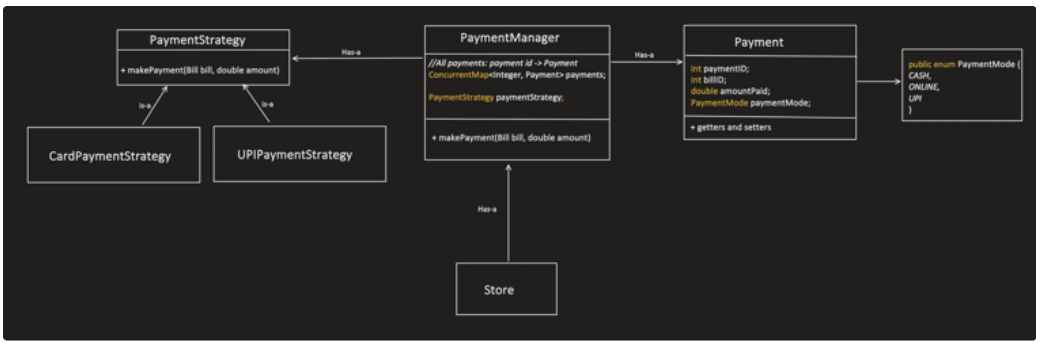
Store, use this BillManager to *generateBill* for the given reservation.



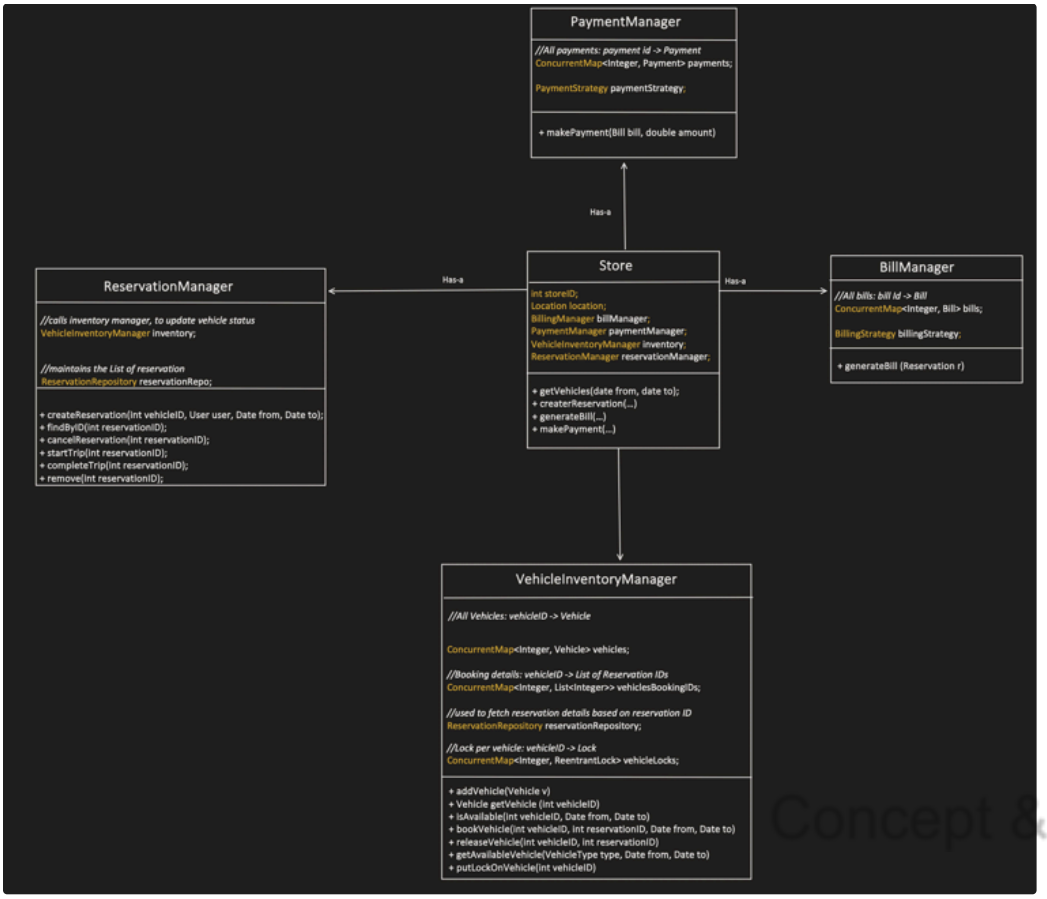
6. `Payment`

- Payment hold info like Bill id like this payment is associated with which bill. We don't need full bill object.
- Also, Payment is just a POJO class which is not smart enough. So we need: PaymentManager, which handles all the payments and their lifecycle.

  Store, use this PaymentManager to *make payment* for the given bill.

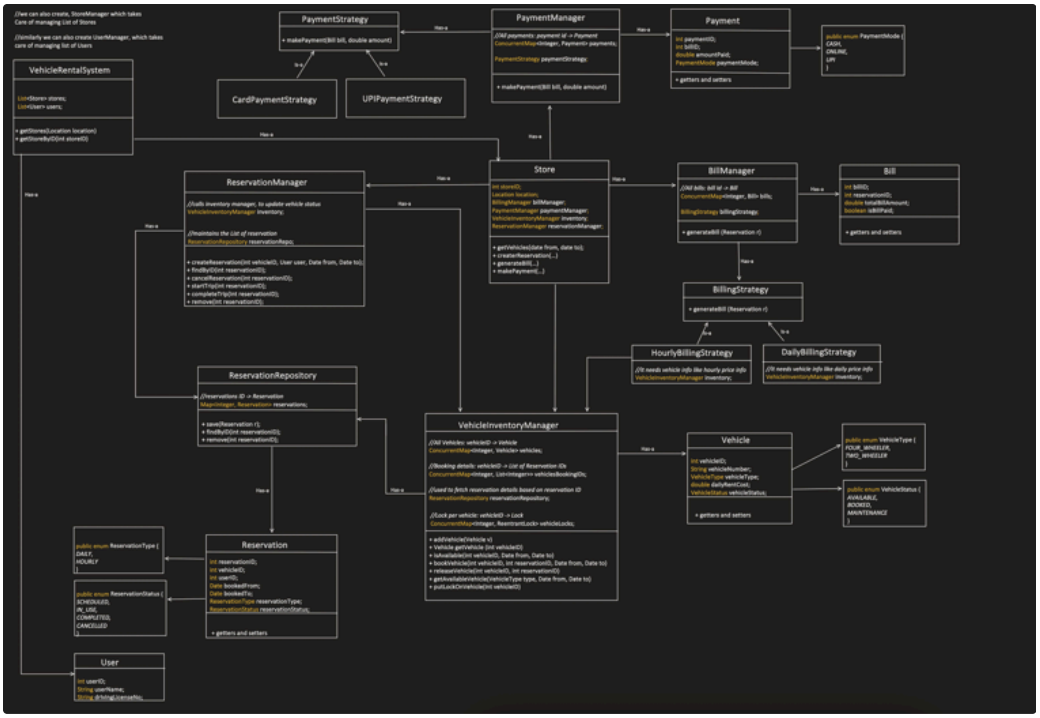


7. `Store`



Concept && Coding

Class Diagram



Implementation

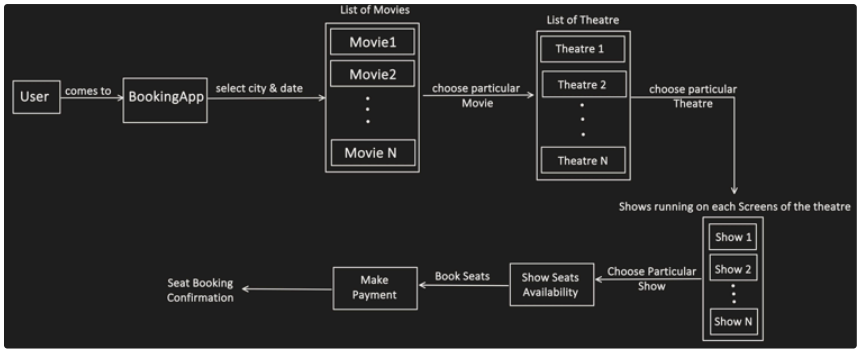
Refer Code Repository for Executable Code → src/main/java/com/conceptcoding/interviewquestions/carrental · main · shrayansh jain / LLD-LowLevelDesign · GitLab

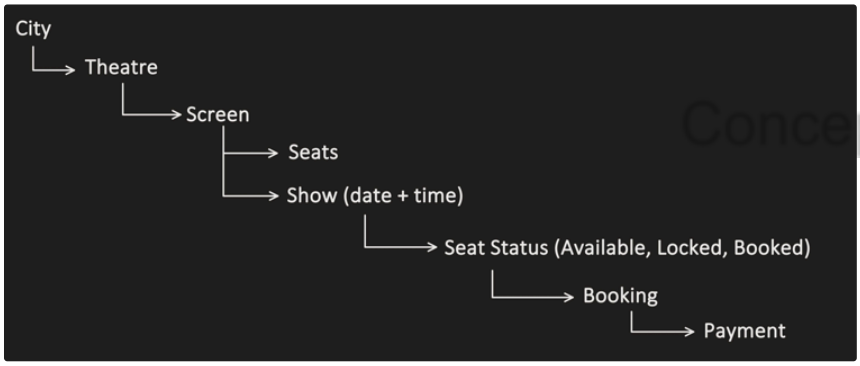
Concept && Coding

Design Movie Ticket Booking App

Reference: Concept&&Coding-By Shrayansh
LLD of BookMyShow (English) | Design MovieTicketBooking

Lets Understand the flow:

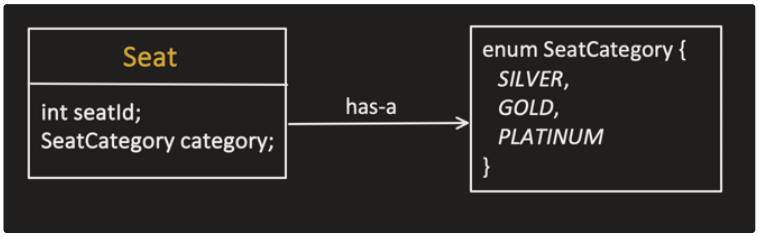




Concept && Coding

UML:

1. Seat

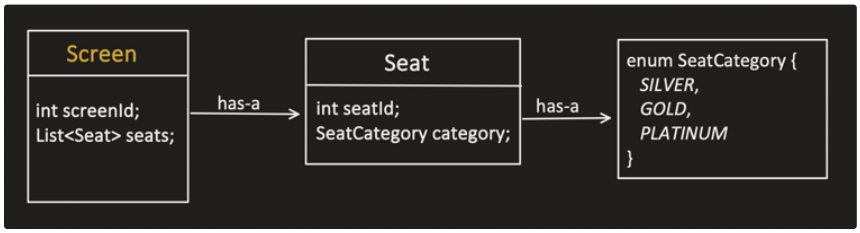



```java
public class Seat {

    private final int seatId;
    private final SeatCategory category;

    public Seat(int seatId, SeatCategory category) {
        this.seatId = seatId;
```

```
        this.category = category;
    }

    public int getSeatId() {
        return seatId;
    }
}
```

```
public enum SeatCategory {
    SILVER,
    GOLD,
    PLATINUM
}
```

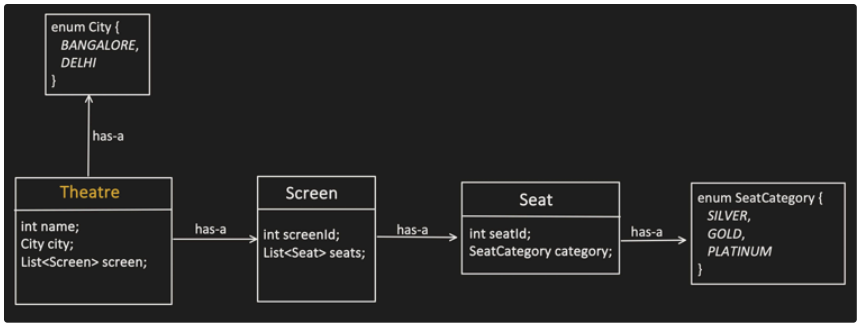
2. Screen




```
public class Screen {

    private final int screenId;
    private final List<Seat> seats;
    private final Map<LocalDate, List<Show>> showsByDate = new
HashMap<>();

    public Screen(int screenId, List<Seat> seats) {
        this.screenId = screenId;
        this.seats = seats;
    }

    public List<Seat> getSeats() {
        return seats;
    }

    public void addShow(Show show) {
        showsByDate
                .computeIfAbsent(show.getShowDate(), d -> new
ArrayList<>())
                .add(show);
    }

    public List<Show> getShows(LocalDate date) {
        return showsByDate.getOrDefault(date, List.of());
    }
}
```

Concept && Coding

3. **Theatre**

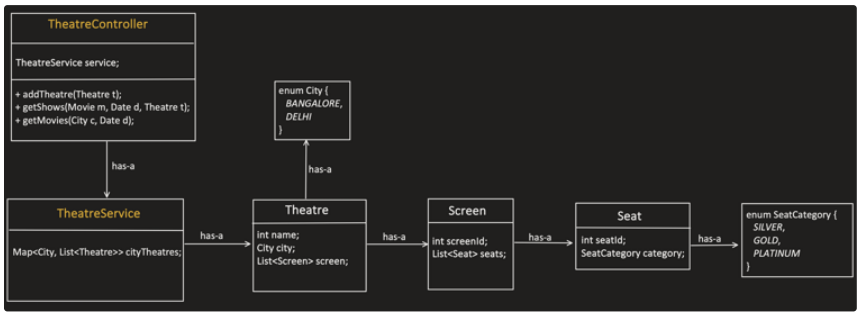



```java
public class Theatre {

    private final String name;
    private final City city;
    private final List<Screen> screens;
    //address info etc.

    public Theatre(String name, City city, List<Screen> screens) {
        this.name = name;
        this.city = city;
        this.screens = screens;
    }

    public City getCity() {
        return city;
    }

    public String getName() {
        return name;
    }

    public List<Screen> getScreens() {
        return screens;
    }
}
```

Concept && Coding

```java
public enum City {
    BANGALORE,
    DELHI
}
```

4. **Theatre Controller**



TheatreController:

```java
public class TheatreController {

    private final TheatreService theatreService;

```

```
    public TheatreController() {
        this.theatreService = new TheatreService();
    }

    public void addTheatre(Theatre theatre) {
        theatreService.addTheatre(theatre);
    }

    public Set<Movie> getMovies(City city, LocalDate date) {
        return theatreService.getMovies(city, date);
    }

    public List<Theatre> getTheatres(City city, Movie movie, LocalDate
date) {
        return theatreService.getTheatres(city, movie, date);
    }

    public List<Show> getShows(Movie movie, LocalDate date, Theatre
theatre) {
        return theatreService.getShows(movie, date, theatre);
    }
}
```

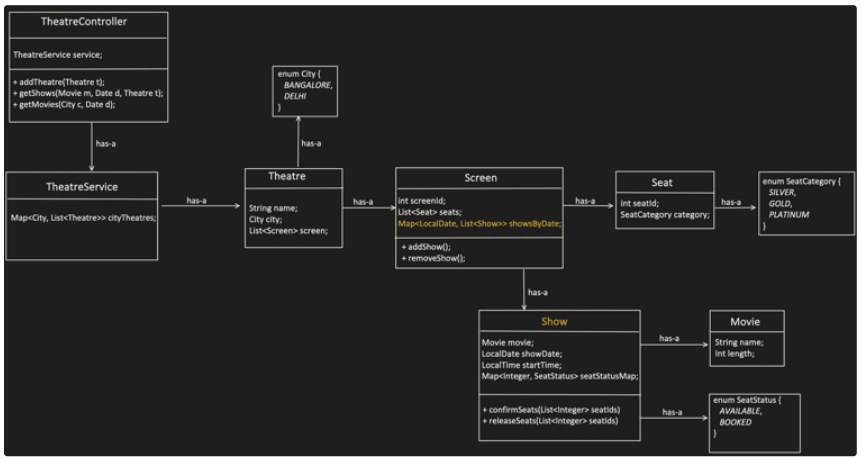
TheatreService:

```
public class TheatreService {

    private final Map<City, List<Theatre>> cityTheatres = new
HashMap<>();

    public void addTheatre(Theatre theatre) {
        cityTheatres
                .computeIfAbsent(theatre.getCity(), c -> new
ArrayList<>())
                .add(theatre);
    }

    public Set<Movie> getMovies(City city, LocalDate date) {
        Set<Movie> movies = new HashSet<>();
        List<Theatre> theatres = cityTheatres.getOrDefault(city,
List.of());

        for (Theatre theatre : theatres) {
            for (Screen screen : theatre.getScreens()) {
                for (Show show : screen.getShows(date)) {
                    movies.add(show.getMovie());
                }
            }
        }
        return movies;
    }

    public List<Theatre> getTheatres(City city, Movie movie, LocalDate
date) {
        List<Theatre> theatres = cityTheatres.getOrDefault(city,
List.of());

        return theatres.stream()
                .filter(t -> t.getScreens().stream()
                        .anyMatch(s -> s.getShows(date).stream()
                                .anyMatch(show ->
show.getMovie().equals(movie))))
                .toList();
    }

    public List<Show> getShows(Movie movie, LocalDate date, Theatre
theatre) {
        List<Show> result = new ArrayList<>();
```

Concept && Coding

```
        for (Screen screen : theatre.getScreens()) {
            for (Show show : screen.getShows(date)) {
                if (show.getMovie().equals(movie)) {
                    result.add(show);
                }
            }
        }
        return result;
    }
}

```

5. Show



Concept && Coding

```
public class Show {

    private final Movie movie;
    private final LocalDate showDate;
    private final LocalTime startTime;

    private final Map<Integer, SeatStatus> seatStatusMap = new
HashMap<>();
    private final Map<Integer, ReentrantLock> seatLocks = new
HashMap<>();

    public Show(Movie movie, Screen screen, LocalDate date, LocalTime
time) {
        this.movie = movie;
        this.showDate = date;
        this.startTime = time;

        for (Seat seat : screen.getSeats()) {
            seatStatusMap.put(seat.getSeatId(), SeatStatus.AVAILABLE);
            seatLocks.put(seat.getSeatId(), new ReentrantLock());
        }
    }

    public Movie getMovie() {
        return movie;
    }

    public LocalDate getShowDate() {
        return showDate;
    }

    public LocalTime getStartTime() {
        return startTime;
    }

```

```
    public boolean lockSeats(List<Integer> seatIds) {
        List<Integer> sorted = new ArrayList<>(seatIds);

        //sorting i am doing to avoid deadlock scenario
        Collections.sort(sorted);

        List<ReentrantLock> acquiredLocks = new ArrayList<>();

        try {
            // Phase 1: acquire all locks
            for (int seatId : sorted) {
                ReentrantLock lock = seatLocks.get(seatId);
                lock.lock();
                acquiredLocks.add(lock);
            }

            // Phase 2: validate availability
            for (int seatId : sorted) {
                if (seatStatusMap.get(seatId) != SeatStatus.AVAILABLE)
{
                    return false;
                }
            }

            // Phase 3: mark LOCKED
            for (int seatId : sorted) {
                seatStatusMap.put(seatId, SeatStatus.LOCKED);
            }

            return true;

        } finally {
            // Phase 4: release locks
            for (ReentrantLock lock : acquiredLocks) {
                lock.unlock();
            }
        }
    }

    public void confirmSeats(List<Integer> seatIds) {
        for (int seatId : seatIds) {
            seatStatusMap.put(seatId, SeatStatus.BOOKED);
        }
    }

    public void releaseSeats(List<Integer> seatIds) {
        for (int seatId : seatIds) {
            seatStatusMap.put(seatId, SeatStatus.AVAILABLE);
        }
    }
}
```

Concept && Coding

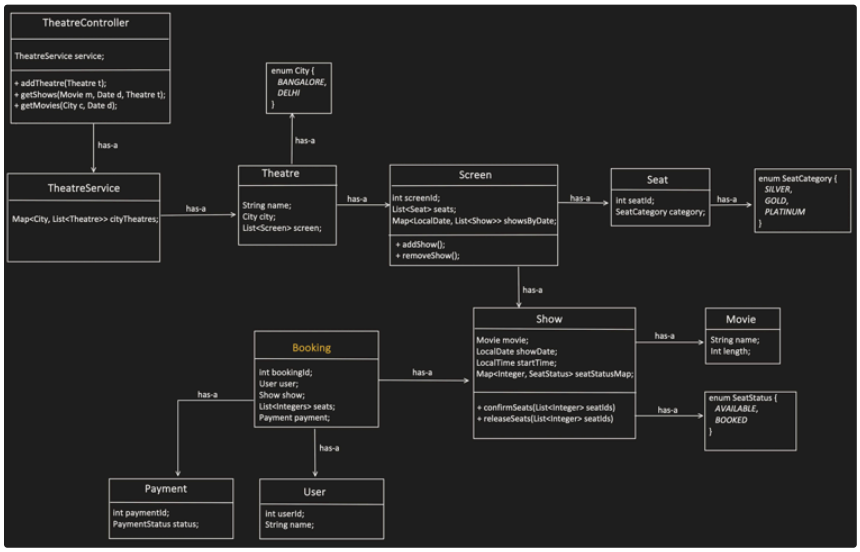
```
public enum SeatStatus {
    AVAILABLE, LOCKED, BOOKED
}
```

Movie:

```
public class Movie {

    private final String name;
    //duration

    public Movie(String name) {
        this.name = name;
    }
```

```
9
10      public String getName() {
11          return name;
12      }
13  }
```

6. **Booking**




```java
public class Booking {

    private final UUID bookingId;
    private final User user;
    private final Show show;
    private final List<Integer> seats;
    private final Payment payment;


    public Booking(User user, Show show, List<Integer> seats, Payment
payment) {
        this.bookingId = UUID.randomUUID();
        this.user = user;
        this.show = show;
        this.seats = seats;
        this.payment = payment;
    }

    public UUID getBookingId() {
        return bookingId;
    }

    public User getUser() {
        return user;
    }

    public Payment getPayment() {
        return payment;
    }
}
```

Concept && Coding

User:

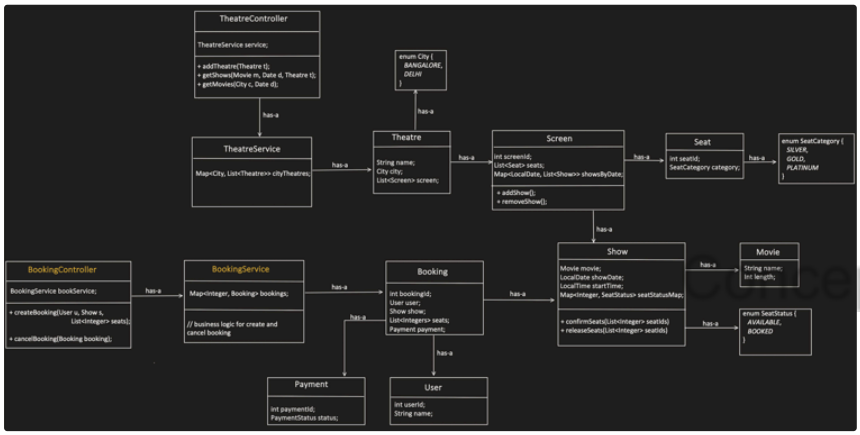
```java
public class User {

    private final String userId;
    private final String name;
```

```
5
6       public User(String userId, String name) {
7           this.userId = userId;
8           this.name = name;
9       }
10  }
11
```

- if Interviewer says, we need to manage the User lifecycle too like add, remove ,delete. Then we can have its own controller like UserController. But here in booking functionality, we are not creating or managing user, so I am skipping exposing the endpoints for managing the lifecycle of User.

- Similarly, I am not creating PaymentController, PaymentService layer for now, but if interviewer want we can add that too.

7. **Booking Controller**



BookingController:

```
1   public class BookingController {
2
3       private final BookingService bookingService;
4
5       public BookingController() {
6           this.bookingService = new BookingService();
7       }
8
9       public Booking createBooking(User user, Show show, List<Integer>
    seats) {
10          Booking booking = bookingService.book(user, show, seats);
11          return booking;
12      }
13
14      public Booking getBooking(UUID bookingId) {
15          return bookingService.getBooking(bookingId);
16      }
17
18      public List<Booking> getBookingsForUser(User user) {
19          return bookingService.getBookingsForUser(user);
20      }
21  }
22
```

BookingService:

```java
public class BookingService {

    private final Map<UUID, Booking> bookings = new HashMap<>();


    public Booking book(User user, Show show, List<Integer> seats) {

        if (!show.lockSeats(seats)) {
            throw new RuntimeException("Seat unavailable");
        }

        //simulated payment flow here, we can invoke Pay method of Payment Controller
        Payment payment = new Payment(PaymentStatus.SUCCESS);

        if (payment.getStatus() == PaymentStatus.SUCCESS) {
            show.confirmSeats(seats);
            Booking booking =  new Booking(user, show, seats, payment);
            bookings.put(booking.getBookingId(), booking);
            return booking;
        } else {
            show.releaseSeats(seats);
            throw new RuntimeException("Payment failed");
        }
    }

    public Booking getBooking(UUID bookingId) {
        return bookings.get(bookingId);
    }

    public List<Booking> getBookingsForUser(User user) {
        return bookings.values()
                .stream()
                .filter(b -> b.getUser().equals(user))
                .toList();
    }
}
```

Concept && Coding

Now lets talk about concurrency, 2 users should not book same seat.

So, what's the right place to put the lock:

1. Show?

It will work, but what if 2 users are booking totally different seats?

U1: S1, S2

U2: S4,S5

If we are putting the lock at Show level, then even users are booking the different seats, need to wait for the lock release.

2. Show Seat level?

This give us higher level of concurrency. But every important point is, we need to avoid deadlock scenario:

U1: S1, S5

U2: S5, S1

Now,:

U1 has acquired the lock on S1

U2 has acquired the lock on S5

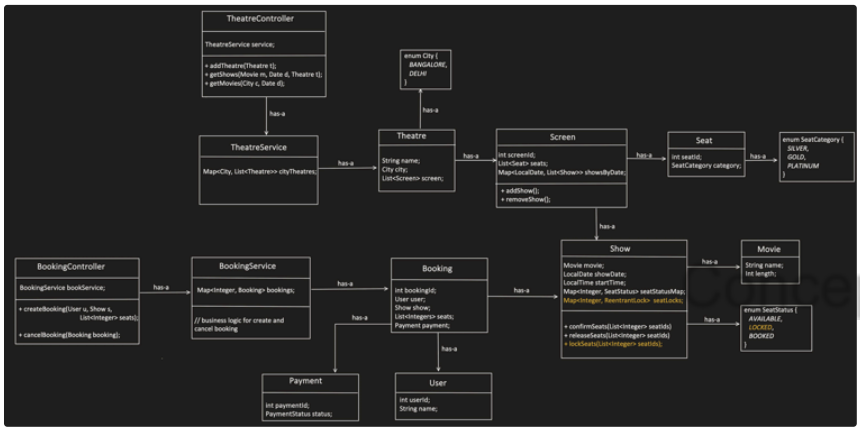
Now both are waiting for each other to release the lock on S5 and S1 respectively. Hence a deadlock scenario.

So, we can easily solve this issue through sorting of the Seat list.

U1: S1, S5 → Sorted → S1, S5

U2: S5, S1 → Sorted → S1, S5

Now, deadlock is not possible.



Client(BookMyShowApp):

```java
public class BookMyShowApp {

    private TheatreController theatreController;
    private BookingController bookingController;

    public static void main(String[] args) {
        BookMyShowApp app = new BookMyShowApp();
        app.initialize();
        app.userFlow();
    }


    private void initialize() {
        theatreController = new TheatreController();
        bookingController = new BookingController();


        /*
         * 1. Create Movies
         */
        Movie baahubali = new Movie("BAAHUBALI");
        Movie avengers = new Movie("AVENGERS");


```

```
        /*
         * 2. Create Theatre -> Screen -> Seats
         */
        Screen inoxScreen1 = new Screen(1, createSeats());
        Theatre inoxTheatreBangalore = new Theatre(
                "INOX",
                City.BANGALORE,
                List.of(inoxScreen1)
        );

        Screen pvrScreen1 = new Screen(1, createSeats());
        Theatre pvrTheatreDelhi = new Theatre(
                "PVR",
                City.DELHI,
                List.of(pvrScreen1)
        );

        theatreController.addTheatre(inoxTheatreBangalore);
        theatreController.addTheatre(pvrTheatreDelhi);


        /*
         * 3. Create Shows
         */
        Show inoxMorningShowToday = new Show(
                baahubali,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(8, 0)
        );

        Show inoxAfternoonShowToday = new Show(
                baahubali,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(15, 0)
        );

        Show inoxEveningShowToday = new Show(
                avengers,
                inoxScreen1,
                LocalDate.now(),
                LocalTime.of(18, 0)
        );


        Show pvrMorningShowTomorrow = new Show(
                baahubali,
                pvrScreen1,
                LocalDate.now().plusDays(1),
                LocalTime.of(9, 0)
        );


        // Attach shows to screens
        inoxScreen1.addShow(inoxMorningShowToday);
        inoxScreen1.addShow(inoxAfternoonShowToday);
        inoxScreen1.addShow(inoxEveningShowToday);
        pvrScreen1.addShow(pvrMorningShowTomorrow);
    }

    /*
     * USER FLOW (END TO END)
     */
    private void userFlow() {

        // User enters system
        User user = new User("U1", "Shrayansh");

        System.out.println("User logged in: Shrayansh");

```

Concept && Coding

```
        // 1. User selects city
        City selectedCity = City.BANGALORE;
        System.out.println("Selected City: " + selectedCity);

        // 2. for specific date, Show movies running in city
        LocalDate selectedDate = LocalDate.now();
        System.out.println("Selected Date: " + selectedDate);

        Set<Movie> movies = theatreController.getMovies(selectedCity,
selectedDate);
        System.out.println("Movies available:");
        movies.forEach(m -> System.out.println(" - " + m.getName()));

        // 3. User selects movie
        Movie selectedMovie = movies.iterator().next(); //selecting
first movie
        System.out.println("Selected Movie: " +
selectedMovie.getName());


        // 4. Show theatres and show times in city
        List<Theatre> theatres =
theatreController.getTheatres(selectedCity, selectedMovie,
selectedDate);
        System.out.println("Theatres available:");
        theatres.forEach(t -> System.out.println(" - " +
t.getName()));

        // 6. User selects theatre
        Theatre selectedTheatre = theatres.get(0);
        System.out.println("Selected Theatre: " +
selectedTheatre.getName());

        // 7. Show running shows for movie + date + theatre
        List<Show> shows =
                theatreController.getShows(
                        selectedMovie,
                        selectedDate,
                        selectedTheatre
                );

        System.out.println("Shows available:");
        shows.forEach(s ->
                System.out.println(" - " + s.getStartTime())
        );

        // 8. User selects show
        Show selectedShow = shows.get(0);
        System.out.println("Selected Show Time: " +
selectedShow.getStartTime());

        // 9. User selects seats
        List<Integer> selectedSeats = List.of(1, 2, 3);
        System.out.println("Selected Seats: " + selectedSeats);

        // 10. Booking + Payment
        Booking booking =
                bookingController.createBooking(
                        user,
                        selectedShow,
                        selectedSeats
                );

        System.out.println("BOOKING SUCCESSFUL");
        System.out.println("Booking ID: " + booking.getBookingId());
    }

    private List<Seat> createSeats() {
        List<Seat> seats = new ArrayList<>();
        for (int i = 1; i <= 20; i++) {
            seats.add(new Seat(i, SeatCategory.SILVER));
```

Concept && Coding

```
        }
        return seats;
    }
}
```

Concept && Coding

Design Elevator System

Resources

- 8. Elevator System, Low Level Design (Hindi) | SDE LLD interview question | Design Elevator System
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/elevator · main · shrayansh jain / LLD-LowLevelDesign · GitLab

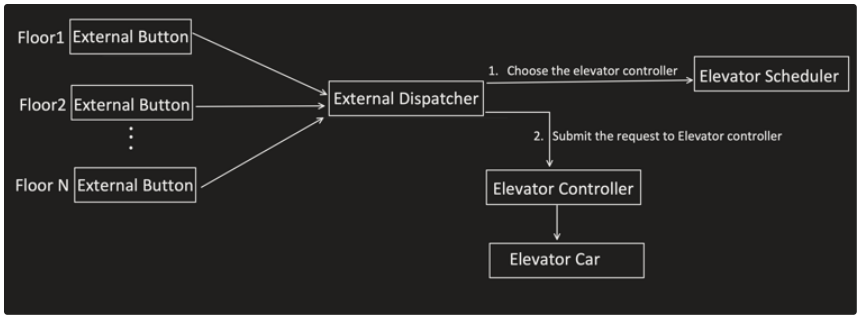
Functionality Requirements:

- A building has multiple **elevators** and multiple **floors**.
- A user can request an elevator **externally** using **Up / Down** buttons at each floor.
- These Up/ Down direction button is used to **choose the best Elevator** to server the request.
- One Elevator is chosen, the floor is added in that particular elevator **bucket list**.
- A user inside an elevator can also press an **internal button** to select destination floor.
- Request generated from elevator Internal button, should always server by the same elevator only.
- Elevators should **remain idle (sleep) when no requests exist**, and **wake** when **new request arrives**.
- Requests should be **ordered by direction**:
  - Going up → visit floors in ascending order
  - Going down → visit floors in descending order

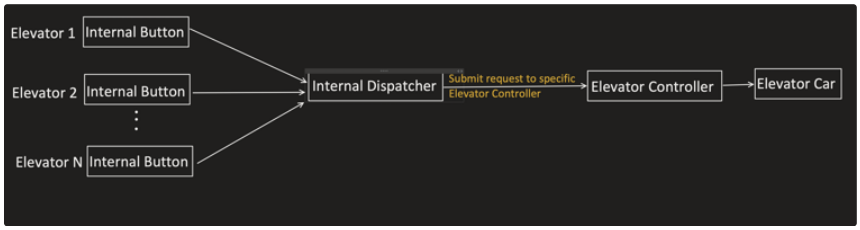
Concept && Coding

Object Identification:

- Building
- Floor
- Elevator - just a POJO
- Elevator Controller - each elevator is controlled by its controller, which manages its requests.
- External Button - Each floor has the External Button
- Internal Button - Each Elevator car has 1 Internal Button
- Elevator Scheduler - Maintains the List of Elevator Controllers + also has Elevator Selection Strategy to choose the best Elevator to serve the request
- External Dispatcher → bridge between External Buttons and Elevator Controller



- Internal Dispatcher → Internal buttons can directly call their Elevator Controller, but sometimes we need to put logging and validation, so its better to have one proxy. Also its follows the same path as external, so its clear too.



Before moving ahead with UML, lets first understand:

1. Elevator Selection Strategy?
   and
2. Elevator moving behavior (i.e requests should be **ordered by direction)**, what does that mean?
   - Going up → visit floors in ascending order
   - Going down → visit floors in descending order

Concept && Coding

Elevator Selection Strategy:

- When External Button is pressed, info which we have is:
  - **Floor**: floor at which user is waiting for the lift
  - **Direction**: whether user wants to go up or down.

Now when there are more than 1 elevator, then we need to select the Elevator based on some strategy like:

- Nearest Elevator : the elevator which is nearest to the user, should handle the request.
- Least Busy Elevator: the elevator which has least number of request, should handle the request

etc.

So, Selection strategy uses :

- Floor info, from where the request has come.
- Direction info, where the user wants to go
- Elevators info like:
  - its request bucket
  - Elevator direction
  - Elevator current floor

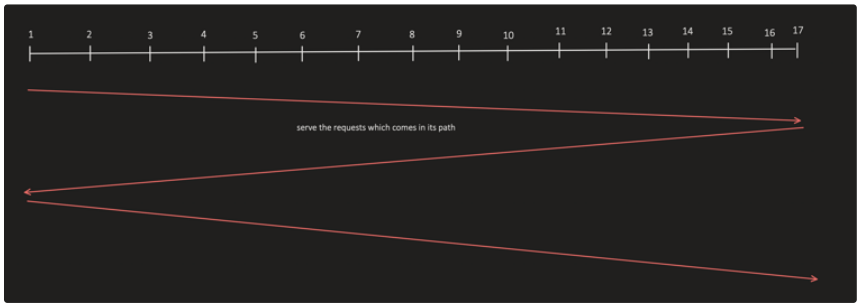
And based on the algorithm, it picks one elevator to serve the request.

Elevator car moving behavior (i.e requests should be ordered by direction):

SCAN Algorithm:

- Head moves in one direction first, servicing the requests along the way.
- Once it reached till the end, it reverses the direction. So it moves End to end.
  EX:

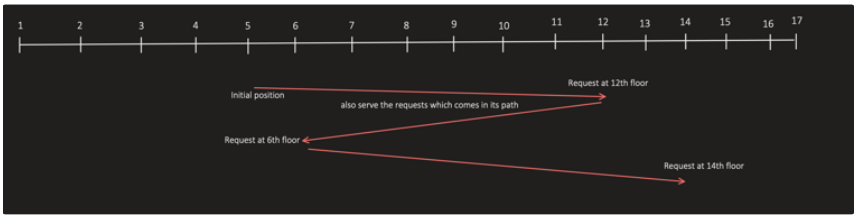
  Elevator going till top floor even if there is no request.



LOOK Algorithm:

Same as SCAN, but with one change that, it Don't go till end, stop when there is no request.

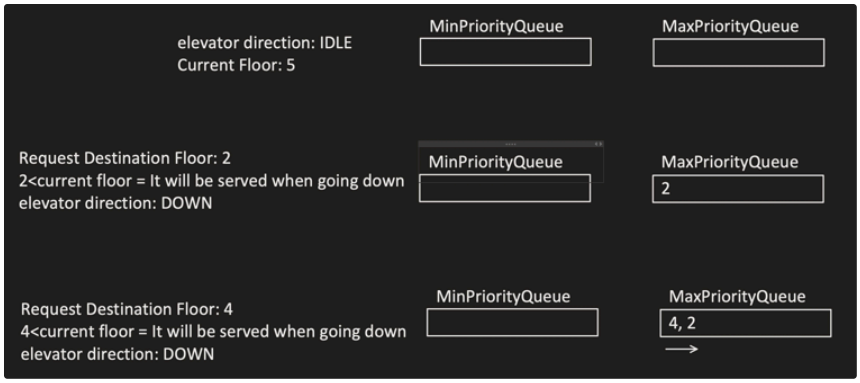
Concept && Coding

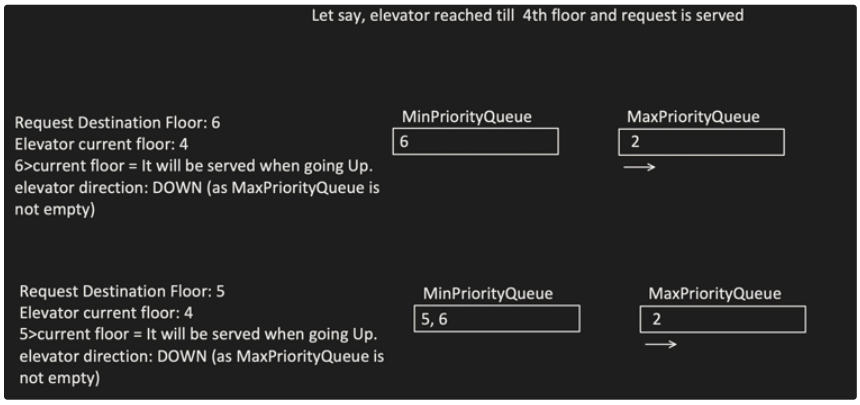


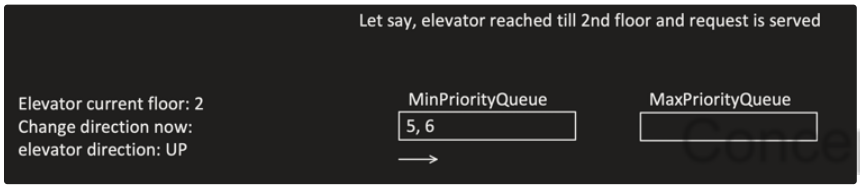
We can implement this LOOK Algorithm using 2 priority queues:

- Min Priority Queue: Used when elevator is going UP, as it need to serve the request in ascending order.
- Max Priority Queue: Used when elevator is going DOWN, as it need to serve the request in descending order.

Example:



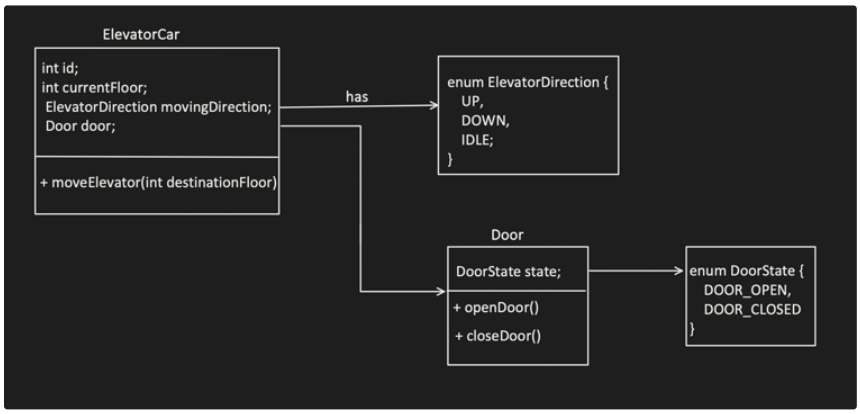




Concept && Coding

UML:

1. **Elevator Car:** An object, which moves to particular destination when asked to.




```java
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class ElevatorCar {

    int id;
    int currentFloor;
```

```
    int nextFloorStoppage;
    ElevatorDirection movingDirection;
    Door door;

    public ElevatorCar(int id) {
        this.id = id;
        currentFloor = 0;
        movingDirection = ElevatorDirection.IDLE;
        door = new Door();
    }

    public void showDisplay() {
        System.out.println("elevator:" + id + " Current floor: " +
currentFloor + " going: " + movingDirection);
    }

    public void moveElevator(int destinationFloor) {
        //this is dump object, so if command has come, to go
particular direction and particular floor, it just move
        //no matter what its current state and floor.

        this.nextFloorStoppage = destinationFloor;
        if (this.currentFloor == nextFloorStoppage) {
            door.openDoor(id);
            return;
        }

        int startFloor = this.currentFloor;
        door.closeDoor(id);
        if(nextFloorStoppage >=currentFloor) {
            movingDirection = ElevatorDirection.UP;
            showDisplay();
            //+1 i am doing bcoz, floor start from 0,1,2.... so if
anyone goes from 1st floor to 2nd, so only 1 floor
            //lift has to move, not 2.
            for (int i = startFloor+1; i<= nextFloorStoppage; i++) {
                try {
                    Thread.sleep(5);
                }catch (Exception e) {

                }
                setCurrentFloor(i);
                showDisplay();
            }
        }
        else  {
            movingDirection = ElevatorDirection.DOWN;

            showDisplay();
            for (int i = startFloor-1; i>= nextFloorStoppage; i--) {
                try {
                    Thread.sleep(5);
                }catch (Exception e) {

                }
                setCurrentFloor(i);
                showDisplay();
            }
        }
        door.openDoor(id);
    }

    public void setCurrentFloor(int currentFloor) {
        this.currentFloor = currentFloor;
    }
}
```

Concept && Coding

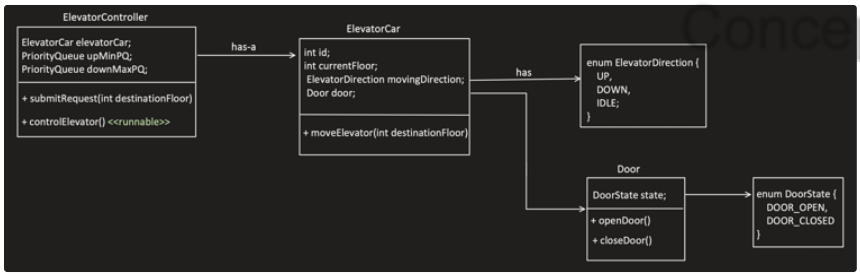
```
public enum ElevatorDirection {
    UP,
```

```
    DOWN,
    IDLE;
}
```

```
public class Door {
    private DoorState doorState;

    Door() {
        doorState = DoorState.DOOR_CLOSED;
    }

    public void openDoor(int id) {
        doorState = DoorState.DOOR_OPEN;
        System.out.println("Opening the Elevator door of elevator:" +
id);
    }

    public void closeDoor(int id) {
        doorState = DoorState.DOOR_CLOSED;
        System.out.println("Closing the Elevator door of elevator:" +
id);
    }

}
```

```
public enum DoorState {
    DOOR_OPEN,
    DOOR_CLOSED
}
```

2. **Elevator Controller**



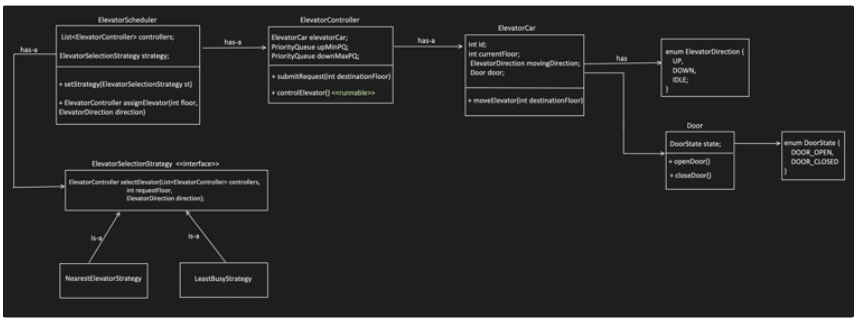
Concept && Coding

```
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.concurrent.PriorityBlockingQueue;

public class ElevatorController implements Runnable {

    PriorityBlockingQueue<Integer> upMinPQ;
    PriorityBlockingQueue<Integer> downMaxPQ;

    ElevatorCar elevatorCar;

    private final Object monitor = new Object();

    ElevatorController(ElevatorCar elevatorCar) {

        this.elevatorCar = elevatorCar;
        upMinPQ = new PriorityBlockingQueue<>();
        downMaxPQ = new PriorityBlockingQueue<>(10, (a, b) -> b - a);
    }

    public void submitRequest(int destinationFloor) {
        enqueueRequest(destinationFloor);
```

```
    }

    private void enqueueRequest(int destinationFloor) {
        System.out.println("Request details-> destinationFloor: " +
destinationFloor + " accepted by elevator:" + elevatorCar.id);

        if (destinationFloor == elevatorCar.nextFloorStoppage){
            return;
        }
        if (destinationFloor >= elevatorCar.nextFloorStoppage) {
            if (!upMinPQ.contains(destinationFloor)) {
                upMinPQ.offer(destinationFloor);
            }
        } else {
            if (!downMaxPQ.contains(destinationFloor)) {
                downMaxPQ.offer(destinationFloor);
            }
        }

        synchronized (monitor) {
            monitor.notify();   // wake elevator thread
        }
    }

    @Override
    public void run() {
        controlElevator();
    }

    public void controlElevator() {

        while (true) {

            //no request, go to sleep
            synchronized (monitor) {
                while (upMinPQ.isEmpty() && downMaxPQ.isEmpty()) {
                    try {
                        System.out.println("elevator:" +
elevatorCar.id + " is IDLE");
                        elevatorCar.movingDirection =
ElevatorDirection.IDLE;
                        monitor.wait(); // sleep until request arrives
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }


            while (!upMinPQ.isEmpty()) {
                int floor = upMinPQ.poll();
                System.out.println("Serving floor: " + floor + " by
elevator:" + elevatorCar.id + " currentFloor: " +
elevatorCar.currentFloor);
                elevatorCar.moveElevator(floor);
            }


            while (!downMaxPQ.isEmpty()) {
                int floor = downMaxPQ.poll();
                System.out.println("Serving floor: " + floor + " by
elevator:" + elevatorCar.id + " currentFloor: " +
elevatorCar.currentFloor);
                elevatorCar.moveElevator(floor);
            }
        }
    }
}
```

Concept && Coding

3. **Elevator Scheduler:**




```java
public class ElevatorScheduler {

    private final List<ElevatorController> controllers;
    private ElevatorSelectionStrategy strategy;

    public ElevatorScheduler(List<ElevatorController> controllers,
                             ElevatorSelectionStrategy strategy) {
        this.controllers = controllers;
        this.strategy = strategy;
    }

    public void setStrategy(ElevatorSelectionStrategy strategy) {
        this.strategy = strategy;
    }

    public ElevatorController assignElevator(int floor,
 ElevatorDirection direction) {
        return strategy.selectElevator(controllers, floor, direction);
    }
}
```

Concept && Coding

```java
public interface ElevatorSelectionStrategy {

   ElevatorController selectElevator(List<ElevatorController>
controllers,
                                     int requestFloor,
                                     ElevatorDirection direction);
}
```

```java
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class NearestElevatorStrategy implements
ElevatorSelectionStrategy {

    @Override
    public ElevatorController selectElevator(List<ElevatorController>
controllers,
                                             int requestFloor,
                                             ElevatorDirection
direction) {

        ElevatorController best = null;
        int minDistance = Integer.MAX_VALUE;

        //1. Pick the one which is going in same direction and minimum
distance from the destination
```

```
        for (ElevatorController controller : controllers) {
            int nextFloorStoppage =
controller.elevatorCar.nextFloorStoppage;

            // Good candidate if moving same direction & not passed
requested floor
            boolean isSameDirectionCandidate =
                    controller.elevatorCar.movingDirection ==
direction &&
                            ((direction == ElevatorDirection.UP &&
nextFloorStoppage <= requestFloor) ||
                                    (direction ==
ElevatorDirection.DOWN && nextFloorStoppage >= requestFloor));

            int dist = Math.abs(nextFloorStoppage - requestFloor);

            if (isSameDirectionCandidate && dist < minDistance) {
                minDistance = dist;
                best = controller;
            }
        }

        // fallback: if not able to choose, pick the idle one
        if (best == null) {
            for (ElevatorController controller : controllers) {
                if(controller.elevatorCar.movingDirection ==
ElevatorDirection.IDLE) {
                    best = controller;
                    break;
                }
            }

            //reached here means, no list is going in same direction
and no lift is IDLE too, then pick any lift
            if(best == null) {
                best = controllers.get(0);
            }
        }
        return best;
    }
}

```

Concept && Coding

```
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class LeastBusyStrategy implements ElevatorSelectionStrategy {

    @Override
    public ElevatorController selectElevator(List<ElevatorController>
controllers,
                                             int requestFloor,
                                             ElevatorDirection
direction) {

        ElevatorController best = null;
        int minLoad = Integer.MAX_VALUE;

        for (ElevatorController controller : controllers) {
            int load = controller.upMinPQ.size() +
                    controller.downMaxPQ.size();

            if (load < minLoad) {
                minLoad = load;
                best = controller;
            }
        }
```

```
        return best;
    }
}

```

4. **Button and dispatchers:**


```
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class ExternalButton {

    private final ExternalDispatcher dispatcher;

    public ExternalButton(ExternalDispatcher dispatcher) {
        this.dispatcher = dispatcher;
    }

    // this direction of external button is only helpful in selecting
the correct elevator
    public void pressButton(int floor, ElevatorDirection direction) {
        dispatcher.submitExternalRequest(floor, direction);
    }
}

```

Concept && Coding

```
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class ExternalDispatcher {

    ElevatorScheduler scheduler;

    public ExternalDispatcher(ElevatorScheduler scheduler) {
        this.scheduler = scheduler;
    }

    public void submitExternalRequest(int floor, ElevatorDirection
direction) {

        ElevatorController controller =
                scheduler.assignElevator(floor, direction);
```

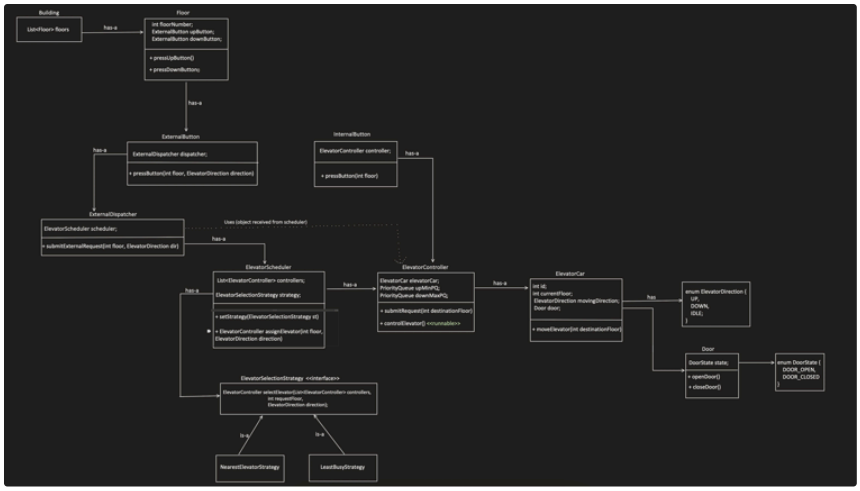
```
        controller.submitRequest(floor);
    }

}

```

```
package com.conceptcoding.interviewquestions.elevator;


public class InternalButton {

    private final ElevatorController controller;

    public InternalButton(ElevatorController controller) {
        this.controller = controller;
    }

    public void pressButton(int destinationFloor) {
        //we can also remove teh Internal dispatcher from mid, but generally say for validation, controller and
        //similar code flow like external button, its good have

        InternalDispatcher.getInstance()
                .submitInternalRequest(destinationFloor, controller);
    }
}
```

```
package com.conceptcoding.interviewquestions.elevator;

public class InternalDispatcher {

    private static InternalDispatcher INSTANCE = new InternalDispatcher();

    private InternalDispatcher() {}

    public static InternalDispatcher getInstance() {
        return INSTANCE;
    }

    // elevatorController is known based on button press origin
    public void submitInternalRequest(int destinationFloor, ElevatorController controller) {
        controller.submitRequest(destinationFloor);
    }
}
```

Concept && Coding

4. **Floors and Building:**




```java
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class Floor {

    int floorNumber;
    ExternalButton upButton;
    ExternalButton downButton;

    public Floor(int floorNumber, ExternalDispatcher dispatcher) {
        this.floorNumber = floorNumber;
        this.upButton = new ExternalButton(dispatcher);
        this.downButton = new ExternalButton(dispatcher);
    }

    public void pressUpButton() {
        upButton.pressButton(floorNumber, ElevatorDirection.UP);
    }

    public void pressDownButton() {
        downButton.pressButton(floorNumber, ElevatorDirection.DOWN);
    }
}
```

Concept && Coding

```java
package com.conceptcoding.interviewquestions.elevator;

import java.util.ArrayList;
import java.util.List;

public class Building {

    List<Floor> floors = new ArrayList<>();

    public Building(int totalFloors, ExternalDispatcher dispatcher) {
        for (int i = 1; i <= totalFloors; i++) {
            floors.add(new Floor(i, dispatcher));
        }
    }

    public Floor getFloor(int floor) {
        return floors.get(floor-1);
    }
}

```

Implementation

Refer Code Repository for executable code → src/main/java/com/conceptcoding/interviewquestions/elevator · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding

Design Parking Lot

Reference:
Video: Design Parking Lot with Complete Implementation (English)
Git link: src/main/java/com/conceptcoding/interviewquestions/parking_lot · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Lets understand the flow first:

- Vehicle arrives at an Entry gate .
  - We can have multiple entry gate

- Based on Vehicle Type, particular parking spot is looked for.
  - Interviewer can ask for flexibility in choosing the parking spot like random, or any specific parking strategy
  - Also interviewer can say, parking lot can have multiple levels.
  - Concurrency should be handled properly, for example:
    - Same parking spot can not be reserved by two different vehicles.
    - for 1 vehicle say two wheeler we should not block the whole parking lot itself

Concept && Coding

- If Parking spot is available, Ticket is generated for particular vehicle
  a. Ticket can have: vehicle no, parking level no, parking spot no and Entry time.

- At Exit gate, cost computation should have happened, Interviewer can ask for different strategies for cost computation like:
  a. Fixed price
  b. Hourly based computation etc.

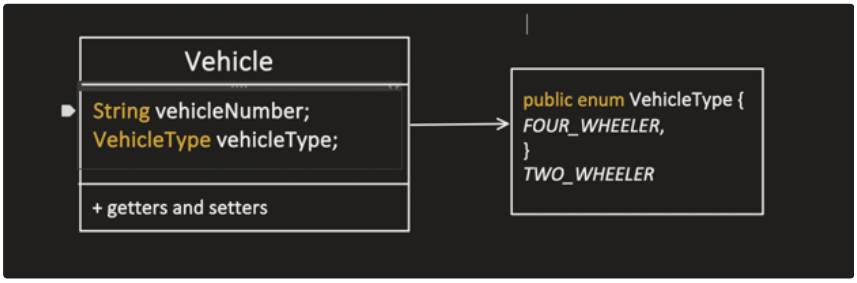
- Payment will be made against the ticket at Exit gate and Parking spot is free again.

Based on above understanding, we identified the below major objects involved in this design:

- Vehicle
- Parking spot
- Parking level
- Entry gate
- Exit Gate
- Ticket
- Payment

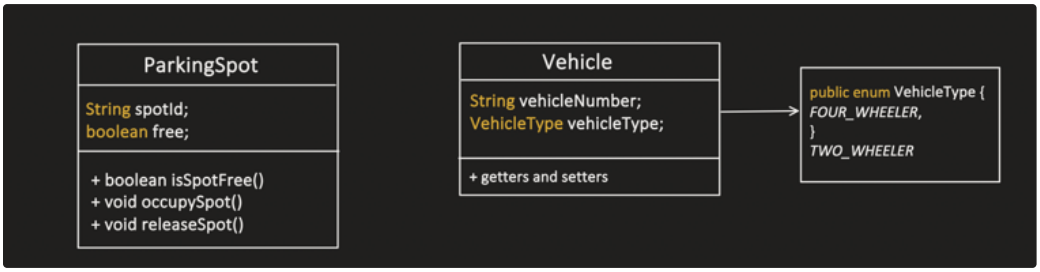
Lets start creating the UML:

1. **Vehicle**



Concept && Coding

2. **ParkingSpot**



ParkingSpot object is not intelligent, its just a POJO.

Also we can have multiple Parking spot (like : 2Wheeler, 4Wheeler, Truck, EV etc.)
We need some other intelligent object,

That's where **ParkingSpotManager** comes into the picture.

3.ParkingSpotManager

Its an intelligent object, which help in managing Parking Spots like add, remove, park, unPark, search free spot etc.

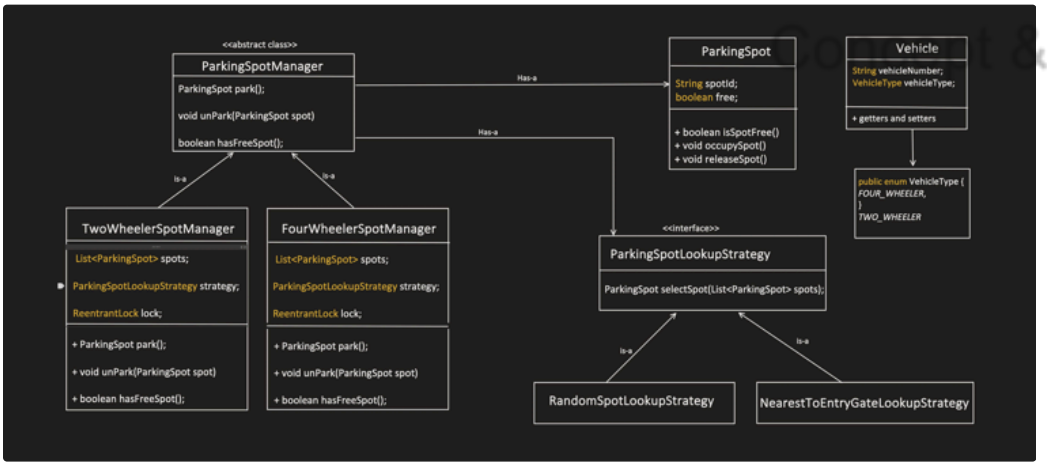
Intention:

We will have different manager to manage different type of parking.

Reasons:

- Each manager has dedicated task to manage their parking spots only.
  - They will search for free space only in their set itself.
  - If they need to put lock during park() and unPark() operation, they can only block their set of spots only.

- Each manager can use different Parking Spot look-up strategy.
  - For ex: 2Wheeler manager is using Random strategy, where as 4Wheeler manager is using NearestToEntraceGate Strategy.



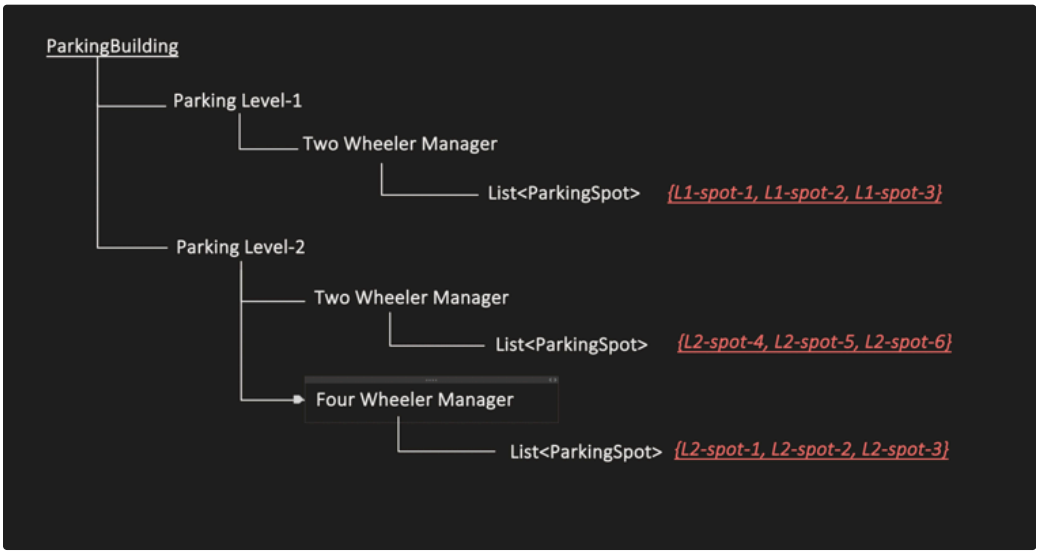
4.ParkingLevel

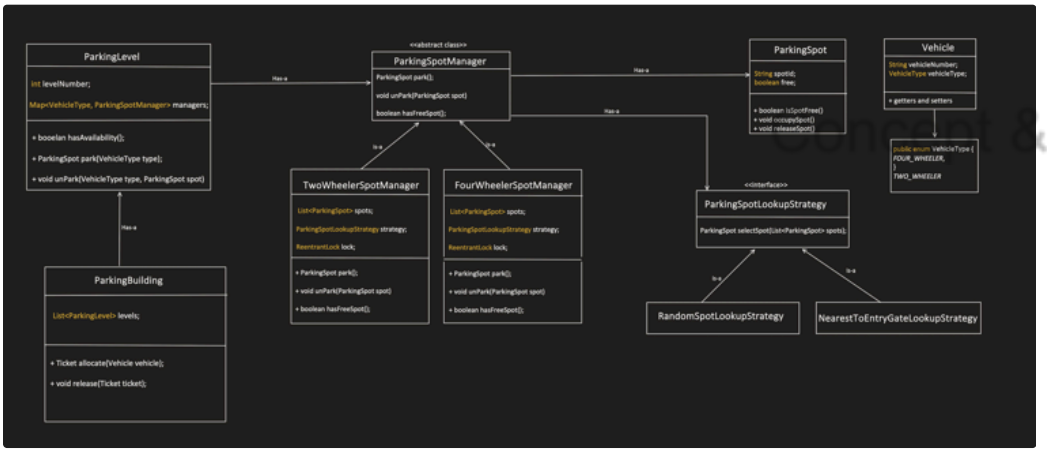
In case of Multi-level parking each level:

- manages their parking independently.

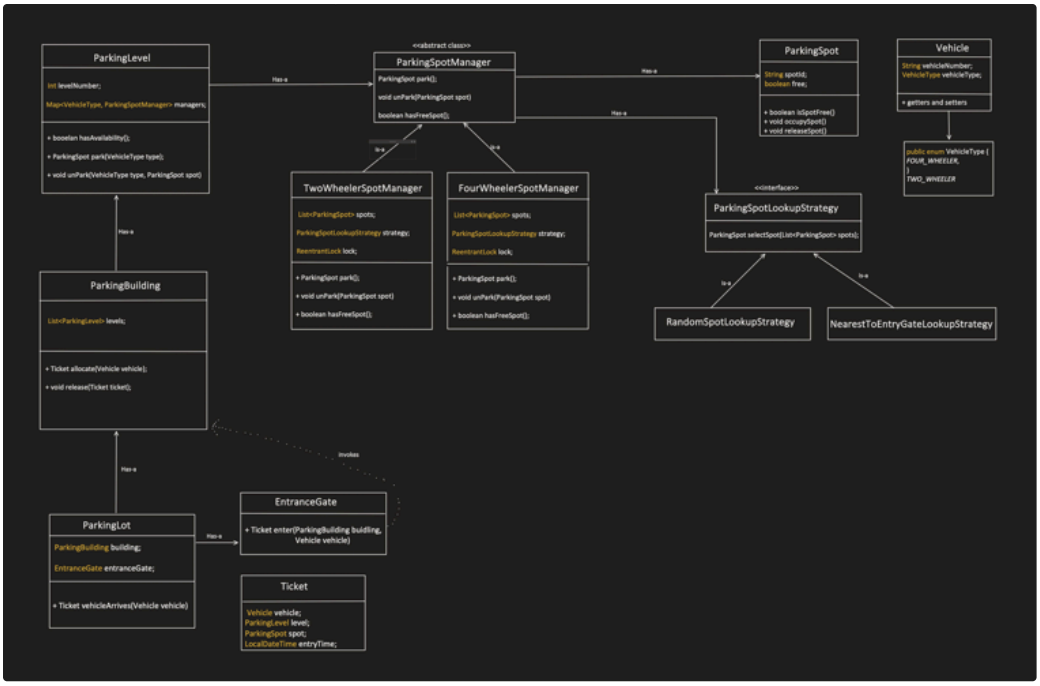
- Each level could support different type of parking, for ex: Level-1 support only 2wheelers, Level-2 supports both 2wheeler and 4Wheelers likewise.

- Also we don't have to lock the whole building parking spots itself, only specific Level and specific Manager (2Wheeler, 4Wheeler etc.) need to be locked during parking and unParking operation.





5.ParkingLot and EntranceGate

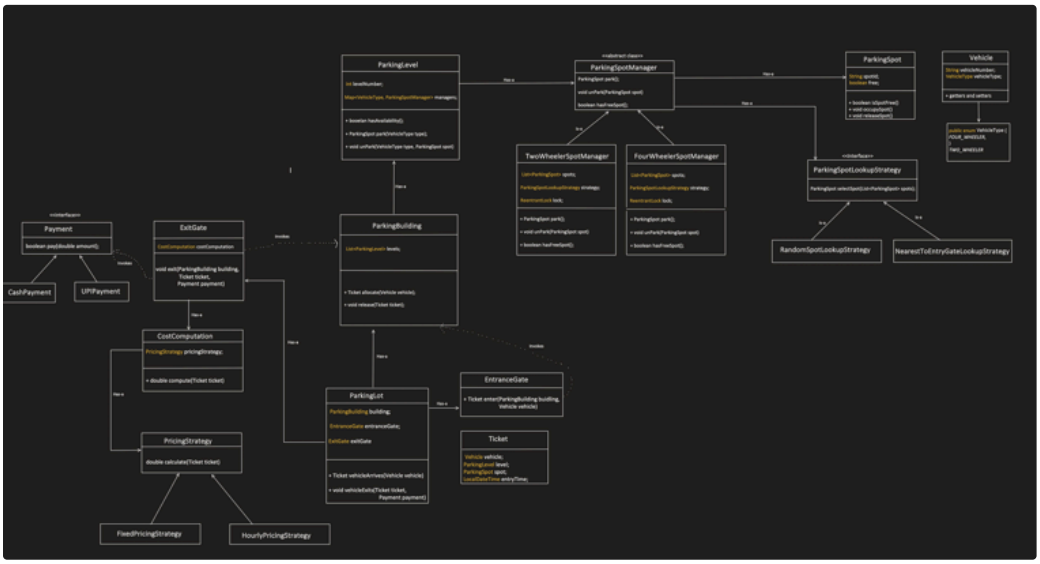


6.Exit Gate

3 things happened at Exit gate:

Concept && Coding

- Based on ticket, Cost computation
- Payment
- On payment success, release of the reserved spot



Reference:

Video: Design Parking Lot with Complete Implementation (English)

Git link: src/main/java/com/conceptcoding/interviewquestions/parking_lot · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding

Design Snake and Ladder Game

Resources

- 11. LLD of Snake and Ladder game (Hindi) | SDE system design interview question, Java implementation
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/snakeNladder · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concepts && Coding

Problem Statement

Design a detailed low-level-design of Snake and Ladder Game including all object-oriented-design patterns and principles discussed earlier.

Overview

A Snake and Ladder Game is a 2-player game(can be extended to have more players by incorporating different winning startergies) where each player takes turns in moving their placement on the grid typically of size 10x10(i.e. 100 cells numbered from [0-99]) by rolling the dice starting from position one(i.e. cell 0).

This game consists of 3 main components:

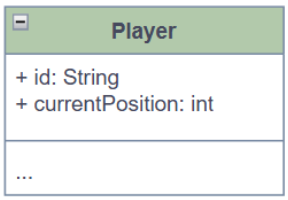
- Dice → Yields a random number between [1-6] which tells the player to move “x” number of cells in forward direction.
- Ladders → Helps the player jump to a higher position.
- Snakes → Steps down the player to a lower position.

Requirement Classification

- Snake and Ladder Game Board grid Size? → Fixed 10x10 i.e. 100 cells
- How many dice? → 1, but it should be scalable. A dice roll function should yield a random number between [1-6].
- Number of Snakes & Ladders → We should be able to dynamically define it.
- Number of Players? → 2 players but configurable.
- Winning Strategy → 2 Player Game, if anyone finishes first(reaches the last position i.e. cell 100 on the grid), the person is the winner and the game is over.
- Game → Acts as a central controller of all the above components.

Components

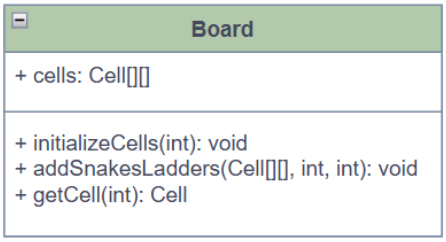
1. `Player`



Concept && Coding

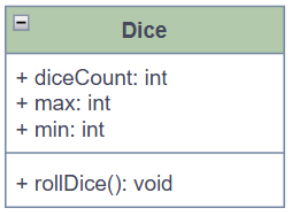
- The `Player` component encapsulates information about the person who choses to play the game.
- Holds variables to save the Name and track the movement during the game.

2. `Board`



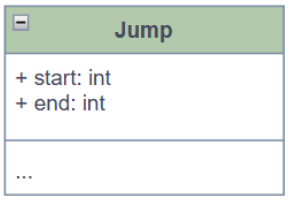
- The `Board` class is composed of cells, a 2D matrix.
- Has methods to initialise, fetch and hold game boards logic.

3. `Dice`



- A `Dice` class holds the max and min value of an inclusive range of numbers that a dice roll can yeild.
- This class is used to simulate the dice roll action using random number generation logic.

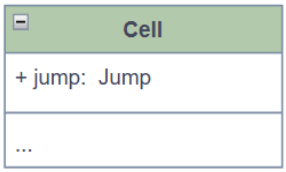
4. `Jump` - Snake and Ladder



Concept && Coding

- The `Jump` class represents the board cell behaviour and holds value of its cell positions.
- A few specific board cells is composed of `Jump` object that simulate Snake and Ladder behaviour when the player acquires that cell upon a dice roll.
- Snake behaviour is implemented using a higher cell position value as start and lower cell position value as end indicating teh player to move backward.
- Ladder value is implemented using using a lower cell position value as start and higher cell position value as end indicating the player to move forward.

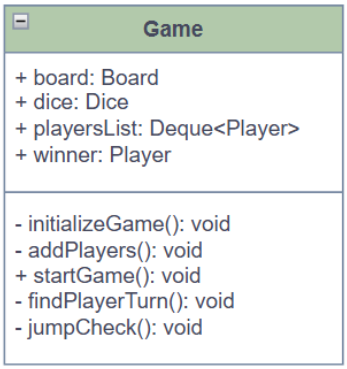
5. `Cell`



- A `Cell` component represents what a game `Board` class is made up of.
- Each `Cell` is composed of a `Jump` object.

- If the `Jump` object is not null, it implies that, this specific cell has a Snake's start or Ladder's start position from which the player either moves backward(if a snake) or moves forward(if a ladder).
- If `Jump` object is null, it means the board cell is not associated with snake or ladder, then the player occupies the new cell position.

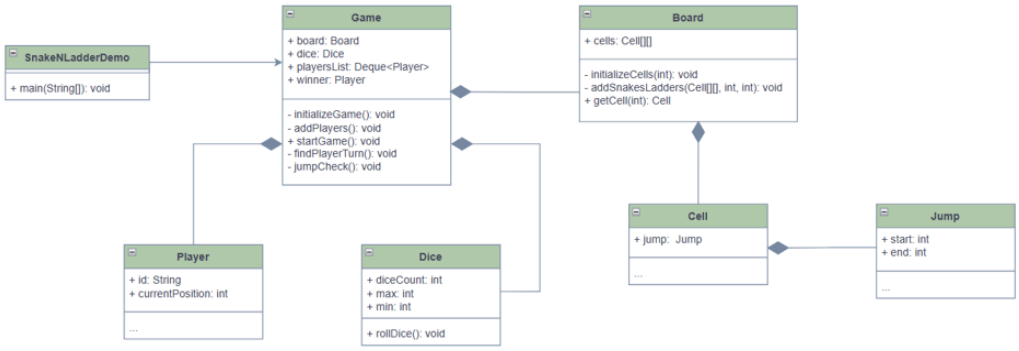
6. `Game`



- The `Game` class acts a central controller that is composed of all necessary components like `Board`, `Dice`, `Players`.
Concept && Coding
- It is responsible for orchestrating the entire Snake and Ladder Game by defining behaviours for various actions that play by the rules established.
- It initialises the `Game` with `Board`, `Dice` and `Players` - the mandatory entities needed to start a game.
- Starts the game, choses player, alternates the turns, rolls dice and moves the current player forward and checks if its new position is associated with a `Jump` behaviour and it is moved further ahead/backward mimicking the Snake/Ladder's behaviour.
- Declares the `Player` who reaches the last board cell position as winner and ends the `Game`.

Class Diagram

We combine all the components discussed above and build a final Class Diagram with all the neccessary class relationships that works as a blueprint to implement an executable solution.



Implementation

Refer to Code Repo for executable code → src/main/java/com/conceptcoding/interviewquestions/snakeNladder · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding

Design a Tic-Tac-Toe Game

Resources

- 7. Design Tic Tac Toe game (Hindi) | Tic-Tac-Toe LLD Java | Low Level Design, System Design
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/tictactoe · main · shrayansh jain / LLD-LowLevelDesign · GitLab

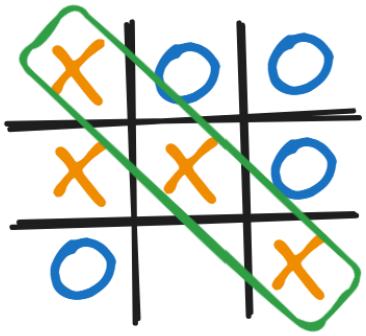
Concept && Coding

Problem Statement

Design a comprehensive 2-player tic-tac-toe game of Xs and Os with the executable code following all design principles, patterns, best practices and guidelines discussed before.

Overview

Tic-tac-toe is a simple two-player paper-and-pencil game. Each player selects a piece before the game and takes turns placing it on the 3x3 grid. A player wins by placing three pieces in a row, column, or diagonal. The game ends when a player wins or when all grid cells are filled, resulting in a draw.



TicTacToe Winner: PlayerX

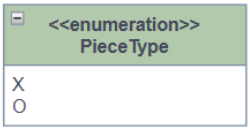
Requirements

- A 3x3 board should be used to play the game.
- 2 Players are marked by the piece they choose to play with - PlayerX and PlayerO
- The game should end when a player wins.
- The game should end when it's a draw(the players are out of free cells to play).
- The game should not allow any invalid moves.

Components

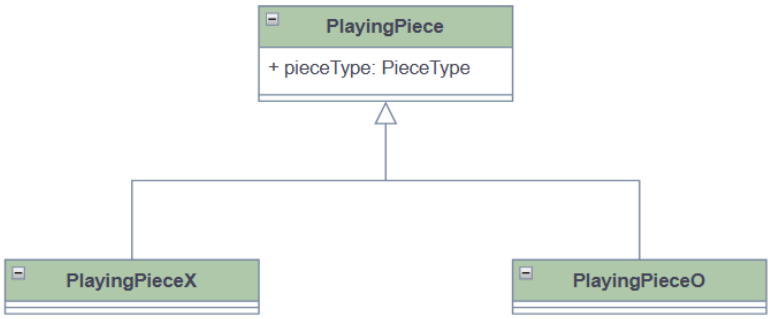
1. `PieceType`

Concept && Coding



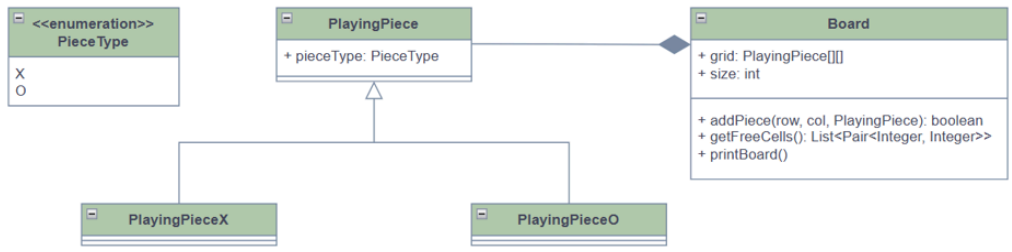
- It's a symbol that is used to represent the cell value.
- Here we are using X and O. We can also choose different symbols.

2. `PlayingPiece`



- The base class represents `PlayingPiece,` used to represent the symbol used.
- Holds a reference to `PieceType` used to represent a piece.
- The concrete classes PlayingPieceX and PlayingPieceO denote specific `PlayingPiece` s associated with the corresponding `PieceType` .

3. `Board`



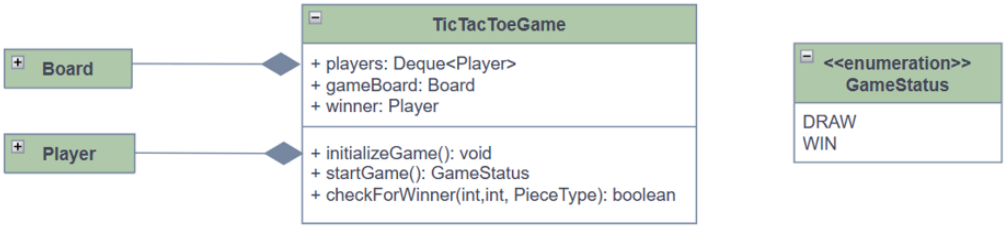
- `Board` is a concrete class that contains a 3x3 matrix of `PlayingPiece` used for playing the game and marking the positions where players place their pieces.
- We can create a grid of any size we want and define corresponding rules to extend the game in future.
- Includes methods to play the game, such as placing a piece, updating the cell value, checking for free cells, and detecting a winning move or game end.

Concept && Coding

4. `Player`

- `Player` is a concrete class that represents a Player in the game.
- It is composed of a `PlayingPiece` type, a symbol that a `Player` chooses to represent at the beginning of the game.

5. `TicTacToeGame`

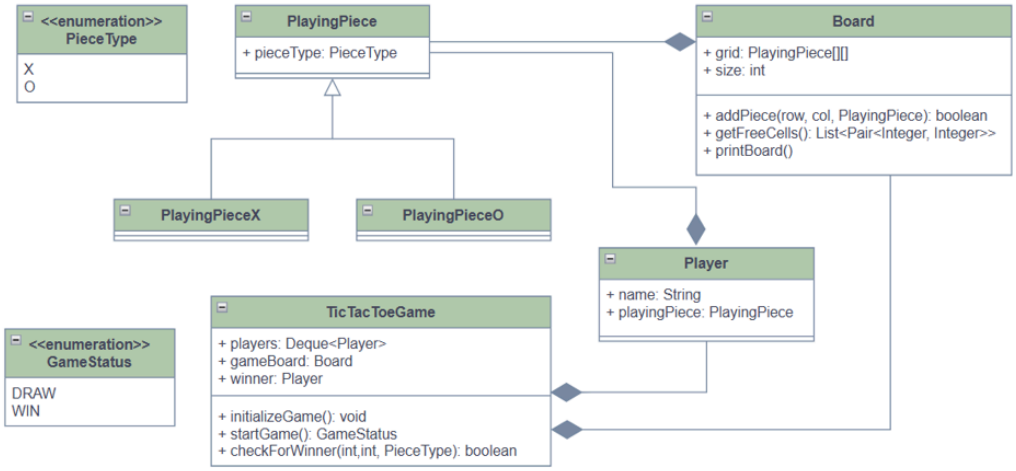


- This concrete class contains core Game control logic.
- This class handles alternating turns between players, validating their moves, checking for a winning move, displaying the board after each step, and declaring the game winner or a draw before ending the game.
- Hence, it is composed of `Players` and a game `Board` reference for orchestrating the `Game` flow.
- Once the Game is over, the appropriate `GameStatus` value is returned.

Class Diagram

We combine the components above to produce a final solution to the TicTacToe Game Design problem.

Concept && Coding



Implementation

Refer Code Repository for Executable Code → src/main/java/com/conceptcoding/interviewquestions/tictactoe · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Output

```
=====>>> TicTacToe Game

     |      |      |
     |      |      |
     |      |      |
Player:Player1 - Please enter [row, column]: 1,0
     |      |      |
X    |      |      |
     |      |      |
Player:Player2 - Please enter [row, column]: 0,0
O    |      |      |
X    |      |      |
     |      |      |
Player:Player1 - Please enter [row, column]: 1,2
O    |      |      |
X    |      | X    |
     |      |      |
Player:Player2 - Please enter [row, column]: 0,2
O    |      | O    |
X    |      | X    |
     |      |      |
Player:Player1 - Please enter [row, column]: 1,1
O    |      | O    |
X    | X    | X    |
     |      |      |

=====>>> GAME OVER: Player1 won the game
Process finished with exit code 0
```

Concept && Coding